

LLM RAG

Technical Project Report

Hybrid retrieval, reranking, grounded local generation, validation, FastAPI and Streamlit

Validated runtime	Python 3.11.11 NVIDIA A30 24 GB PyTorch 2.11.0+cu128
Generation model	Qwen2.5-3B-Instruct, local, CUDA, bfloat16
Retrieval	Dense FAISS + BM25 + RRF + cross-encoder reranking
Serving	FastAPI backend + Streamlit frontend
Evaluation	11-query retrieval holdout + 6-case end-to-end generation evaluation

Scope note: this is a hands-on end-to-end portfolio project designed to demonstrate production-oriented engineering practices. It is not presented as long-term commercial production experience.

September 2026

1. Executive summary

This project implements and validates a complete local Retrieval-Augmented Generation (RAG) system over a small, curated research-paper corpus. The work was intentionally scoped as a serious engineering portfolio project: the objective was not to add as many technologies as possible, but to demonstrate a coherent pipeline that can be inspected, tested, evaluated, served through an API, used through a frontend, and reproduced in the validated compute environment.

The system combines dense FAISS retrieval, BM25 sparse retrieval, Reciprocal Rank Fusion (RRF), cross-encoder reranking, LlamaIndex integration, local generation with Qwen2.5-3B-Instruct, deterministic citation and numeric-grounding checks, semantic groundedness validation, bounded correction attempts, and fail-closed behavior when hard validation rules are not satisfied.

What was achieved

- A working end-to-end path from user query to Streamlit frontend, FastAPI backend, hybrid retrieval, reranking, local GPU generation, validation, and cited response.
- A real HTTP smoke test of FastAPI against the RAG stack, including /health and /query.
- Structured JSON request logging with status and request latency.
- Input validation through Pydantic/FastAPI, including HTTP 422 for invalid empty queries.
- A lightweight automated test suite with 21 passing tests and no failures in the final validated run.
- A retrieval holdout comparing Dense, BM25, Hybrid RRF, and Hybrid RRF + reranker.
- An end-to-end generation evaluation that makes both strengths and weaknesses visible rather than hiding them.
- A reproducible cluster workflow validated on an NVIDIA A30 GPU under SLURM.

2. Problem, objective, and scope

The project addresses a common RAG engineering problem: retrieving useful evidence from a technical document collection, generating a concise answer from that evidence, and exposing enough provenance and validation metadata to make the result inspectable. The application is not intended to answer arbitrary questions; it is deliberately constrained to a curated corpus of five research papers.

Curated corpus

1. ColBERT
2. Dense Passage Retrieval for Open-Domain Question Answering
3. RAG for LLMs: A Survey
4. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
5. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks

The small corpus is a deliberate limitation of the current portfolio version. It makes the evaluation and behavior easier to inspect, but it does not represent the scale or heterogeneity of a production enterprise corpus.

LLM RAG

Hybrid retrieval, reranking, grounded generation and validation.

Ask technical questions across a curated corpus of five research papers

Explore topics such as dense and sparse retrieval, passage reranking, sentence embeddings, RAG architectures and retrieval evaluation.

Pipeline: Dense retrieval (FAISS) + BM25 → Reciprocal Rank Fusion (RRF) → cross-encoder reranking → local LLM generation → citation, numeric grounding and groundedness validation.

Question

Ask a question about the indexed documents...

Run RAG query

Figure 1. Final Streamlit interface. The UI states the five-paper scope and the implemented technical pipeline.

3. System architecture

The design keeps the retrieval logic in the project itself and uses LlamaIndex as an adapter/query-engine integration layer. This avoids duplicating retrieval work: LlamaIndex performs retrieval once, converts the retrieved results into NodeWithScore objects, and then passes those results into the existing RAG pipeline for generation and validation.

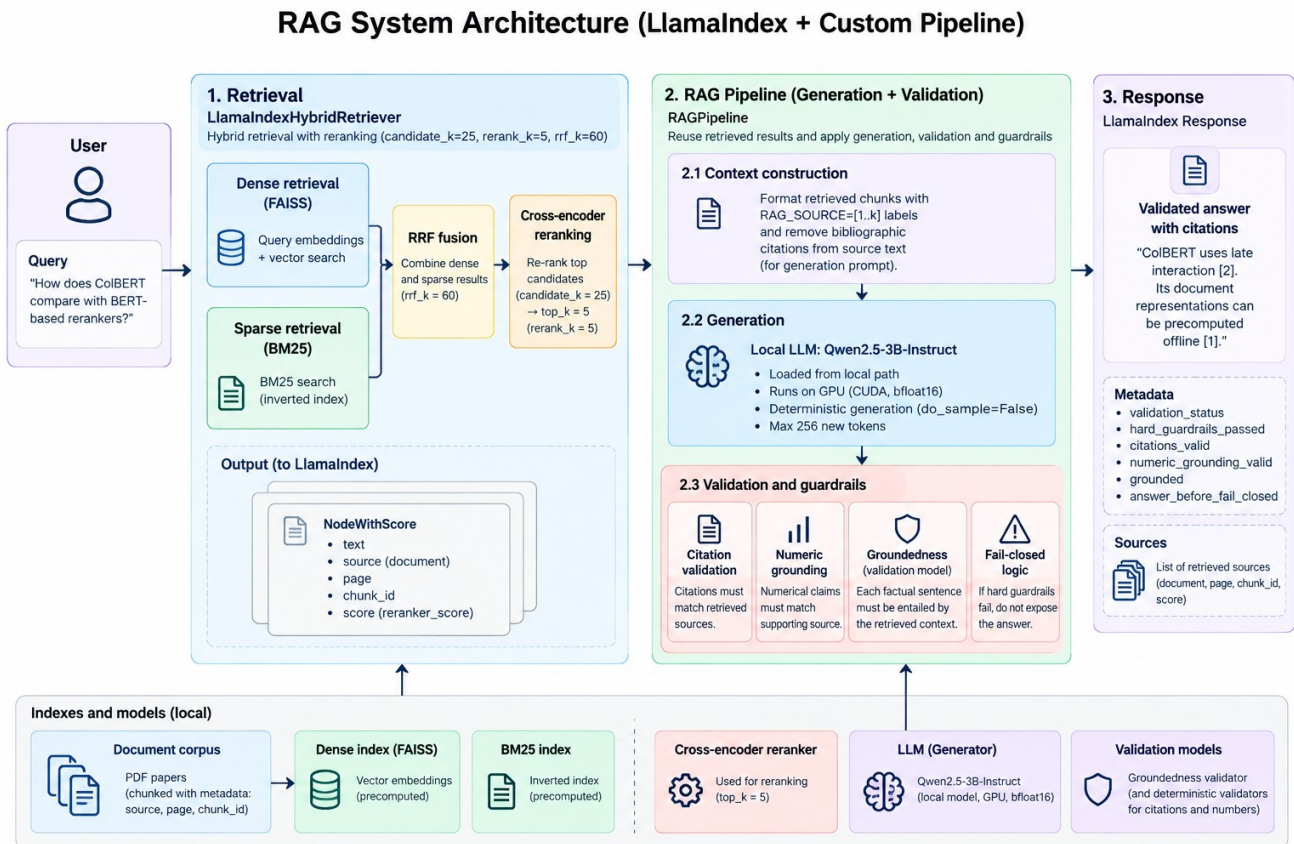


Figure 2. End-to-end RAG architecture used in the validated system.

Runtime query path

- 6. The user submits a technical question in Streamlit.
- 7. The frontend sends the question to FastAPI POST /query.
- 8. LlamaIndexRAGQueryEngine invokes the hybrid retrieval adapter.
- 9. The retrieval pipeline combines dense FAISS and BM25 candidates.
- 10. RRF fuses the candidate rankings.
- 11. A cross-encoder reranker produces the final evidence ranking.
- 12. The retrieved nodes are converted into the project result format.
- 13. RAGPipeline builds a source-labelled context and generates a deterministic answer with Qwen2.5-3B-Instruct.
- 14. Citation, numeric-grounding, and semantic groundedness validators are applied.
- 15. If deterministic hard guardrails fail after the bounded correction path, the pipeline fails closed instead of exposing the candidate answer.
- 16. FastAPI returns the answer, validation metadata, and source metadata to the frontend.

4. Retrieval methodology

The retrieval stack deliberately combines complementary sparse and dense methods. BM25 captures lexical overlap and exact terminology, while dense retrieval captures semantic similarity. Reciprocal Rank Fusion combines both ranked lists without requiring score calibration, and the cross-encoder reranker then applies a more expensive query-document interaction only to a limited candidate set.

candidate_k	25 candidate items before final reranking
rerank_k	5 final evidence items
rrf_k	60
Dense index	FAISS
Sparse retrieval	BM25
Final ranking	Cross-encoder reranker

LlamaIndex integration

LlamaIndex does not replace the project retrieval stack. The adapter translates the project dictionaries to LlamaIndex NodeWithScore objects and then the query engine converts them back into the project result format for generation. This lets the system demonstrate LlamaIndex interoperability while preserving the original retrieval implementation and evaluation path.

5. Generation and validation methodology

The generator uses a local Qwen2.5-3B-Instruct model. In the canonical cluster runtime it requires CUDA, loads local model files only, uses bfloat16 weights, runs in inference mode, and disables sampling for deterministic generation.

Grounding controls

- Generation context is explicitly labelled with RAG source identifiers.
- The prompt requires citations on factual sentences and prohibits invented citation numbers.
- Bibliographic-style numbers inside source text are sanitized from the generation prompt to reduce confusion with RAG citations.
- Citation validation checks that answer citations refer to available RAG sources.
- Numeric-grounding validation checks explicit numbers against cited evidence.
- Semantic groundedness validation evaluates whether generated claims are entailed by retrieved evidence.
- One bounded correction path is available for deterministic failures rather than unbounded self-repair loops.
- Hard deterministic validation failure triggers fail-closed output.

The hard guardrails are citation validity and numeric-grounding validity. Semantic groundedness is exposed separately: an answer can pass deterministic checks while receiving a soft groundedness warning. This distinction is useful because it separates mechanically verifiable constraints from the stricter semantic entailment model.

6. Serving, frontend, and engineering practices

FastAPI exposes a health endpoint and a typed query endpoint. Request models enforce a non-empty query with a maximum length, and the API returns a structured response containing the answer, validation state, and sources. The expensive RAG query engine is initialized once during application lifespan rather than being reloaded for every request.

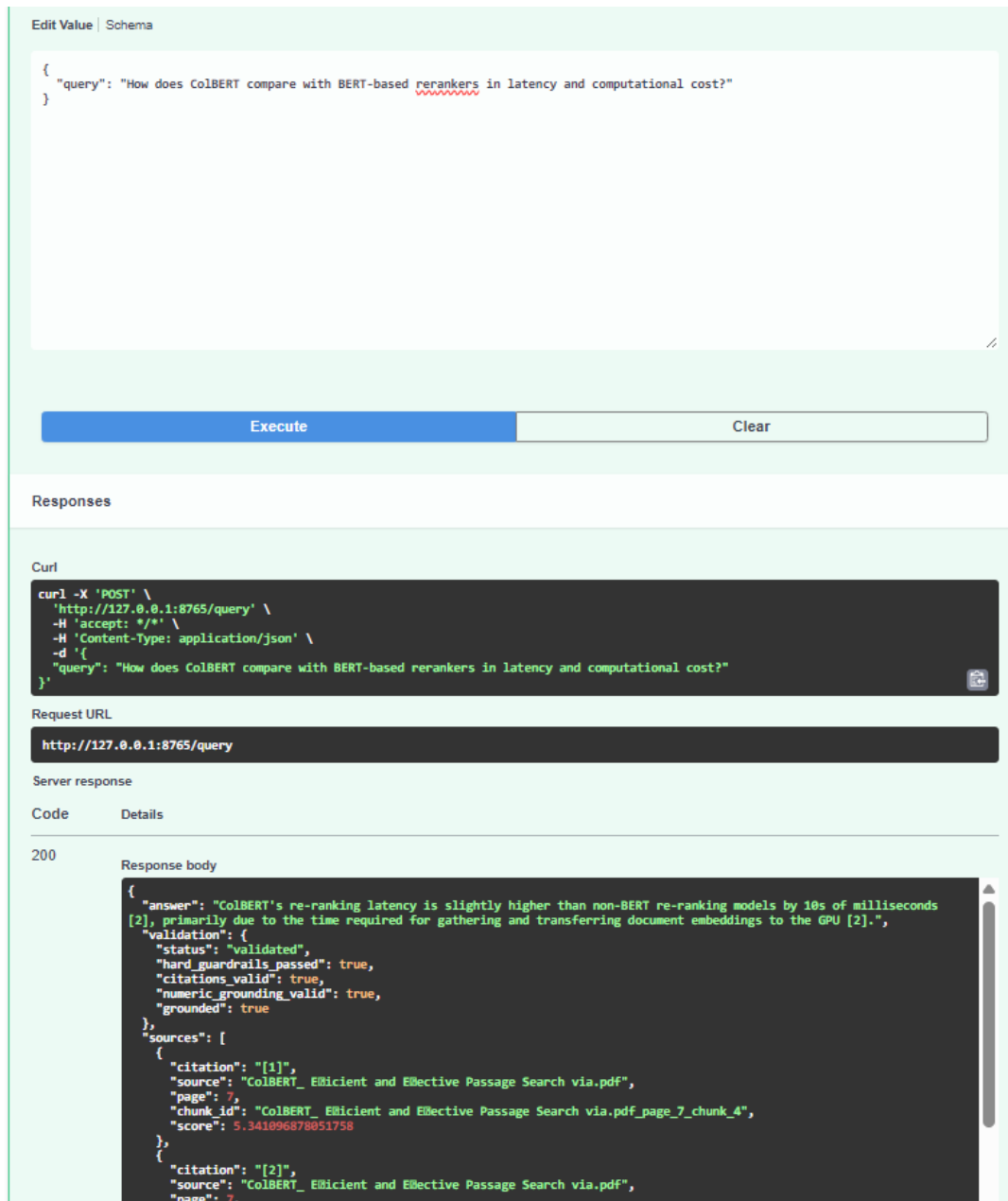


Figure 3. FastAPI Swagger response for a real POST /query request, including answer, validation metadata, and retrieved sources.

Input validation

Pydantic validation rejects an empty query before the RAG engine runs. This behavior was verified through Swagger with HTTP 422 and an explicit string-too-short error.

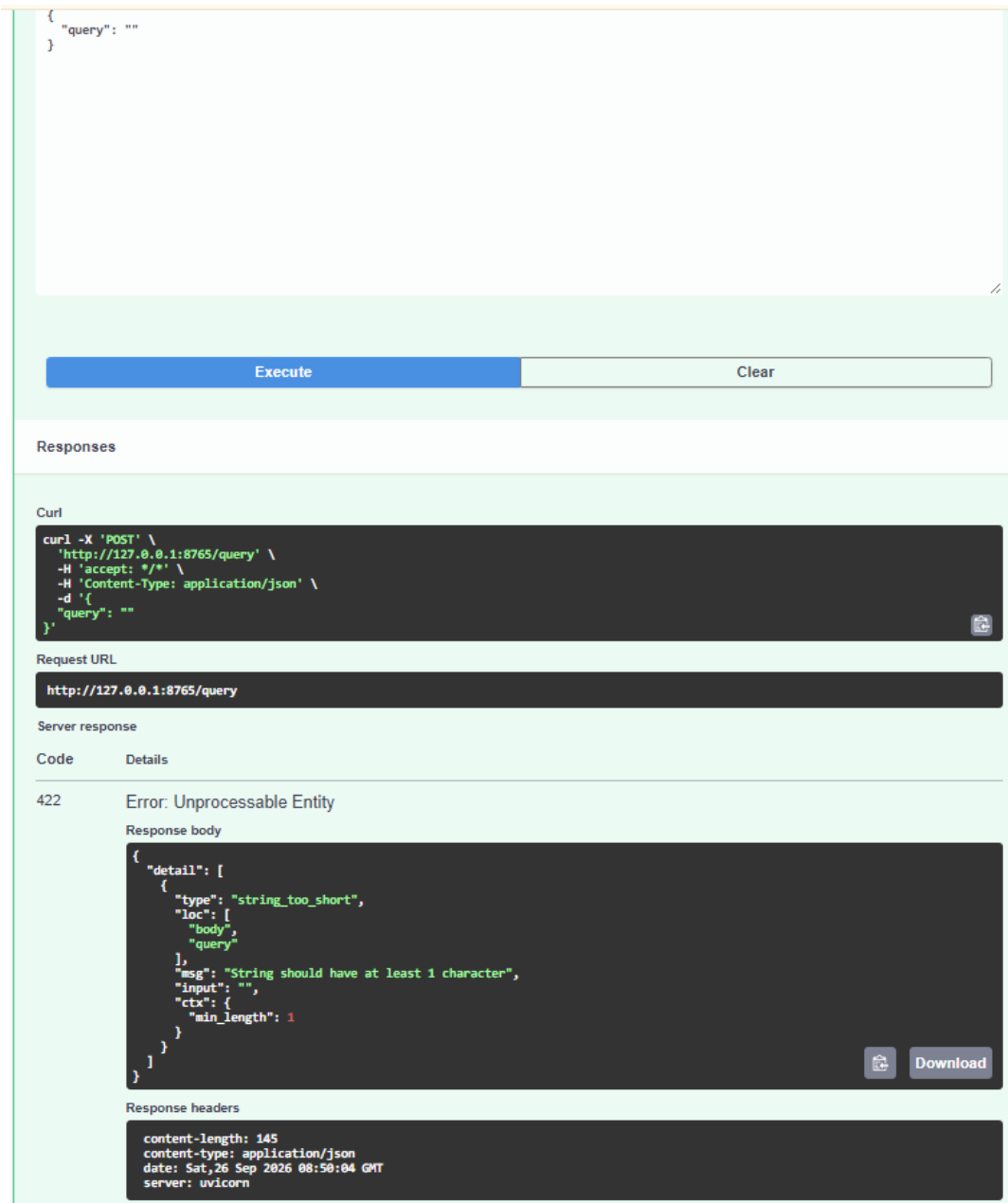


Figure 4. HTTP 422 input validation for an empty query.

Structured logging

The application adds JSON-formatted request logging on top of the standard Uvicorn access logs. Each completed request records a UTC timestamp, log level, logger, event name, HTTP method, request path, status code, and request duration in milliseconds. This provides lightweight observability without claiming a full external monitoring stack.

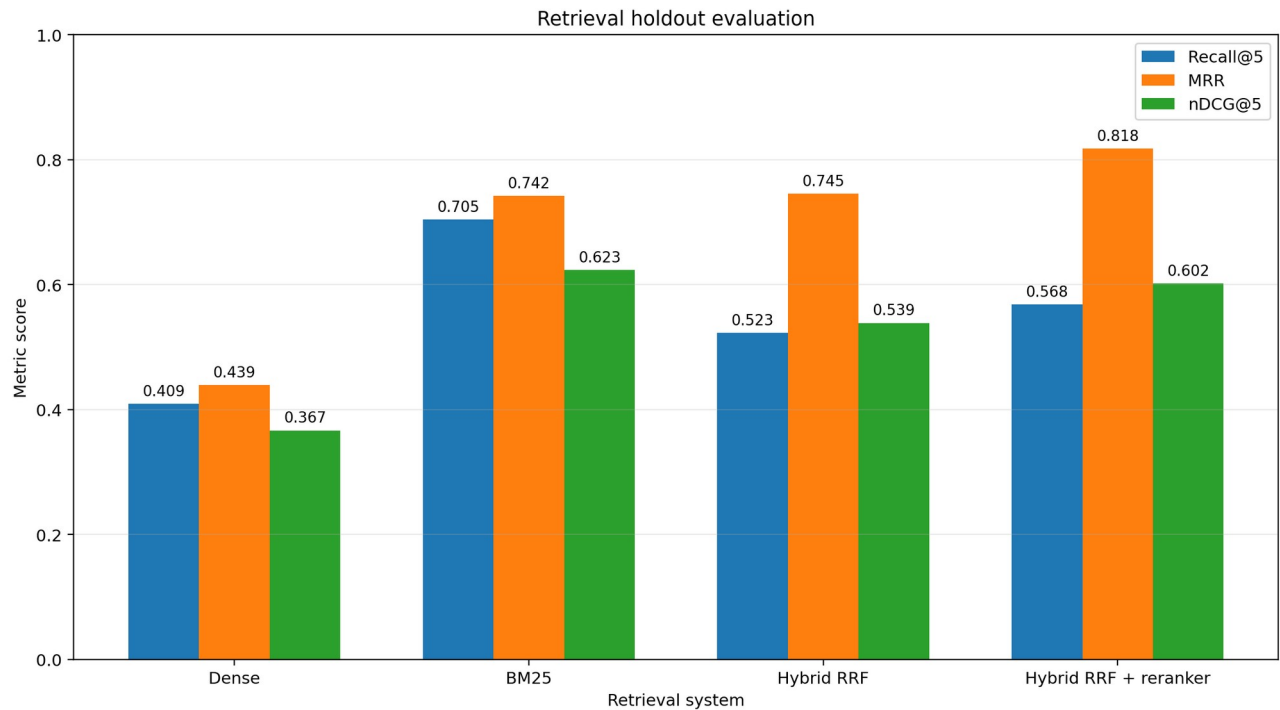
```
Loading weights: 100%|
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8765 (Press CTRL+C to quit)
INFO: 127.0.0.1:52364 - "GET / HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:52364 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:42786 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:36056 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:41556 - "GET /health HTTP/1.1" 200 OK
Batches: 100%|
INFO: 127.0.0.1:41562 - "POST /query HTTP/1.1" 200 OK
INFO: 127.0.0.1:53676 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:53676 - "GET /openapi.json HTTP/1.1" 200 OK
INFO: 127.0.0.1:53688 - "GET /health HTTP/1.1" 200 OK
INFO: 127.0.0.1:53688 - "GET /favicon.ico HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:34332 - "GET /docs HTTP/1.1" 200 OK
INFO: 127.0.0.1:34332 - "GET /openapi.json HTTP/1.1" 200 OK
Batches: 100%|
INFO: 127.0.0.1:34340 - "POST /query HTTP/1.1" 200 OK
```

Figure 5. Uvicorn runtime activity during real health, documentation, and query requests.

7. Retrieval evaluation

Retrieval was evaluated on a frozen holdout of 11 queries with top_k=5. Four systems were compared: dense retrieval, BM25, Hybrid RRF, and Hybrid RRF with reranking.

System	Recall@5	MRR	nDCG@5
Dense	0.409	0.439	0.367
BM25	0.705	0.742	0.623
Hybrid RRF	0.523	0.745	0.539
Hybrid RRF + reranker	0.568	0.818	0.602



Frozen holdout: 11 queries · top_k=5 · candidate_k=25 · rrf_k=60

Figure 6. Aggregate retrieval holdout metrics.

Interpretation

The result is intentionally not reduced to a single "winner". BM25 produced the highest aggregate Recall@5 and nDCG@5 on this small holdout, while the reranked hybrid system produced the highest MRR. Relative to Hybrid RRF alone, adding the reranker improved all three aggregate metrics. The evaluation therefore supports the value of reranking, while also showing that sparse lexical retrieval remains highly competitive on this particular corpus and query set.

8. End-to-end generation evaluation

Six end-to-end cases were evaluated using the actual retrieval and generation path. The purpose was not merely to count successful answers, but to observe how retrieval quality, citation compliance, numeric grounding, semantic groundedness, and hard guardrails interact.

Metric	Result
Mean retrieval Recall@5	0.583
Citation-valid rate	0.833
Numeric-grounding-valid rate	0.833
Grounded rate	0.167
Hard-guardrail pass rate	0.833

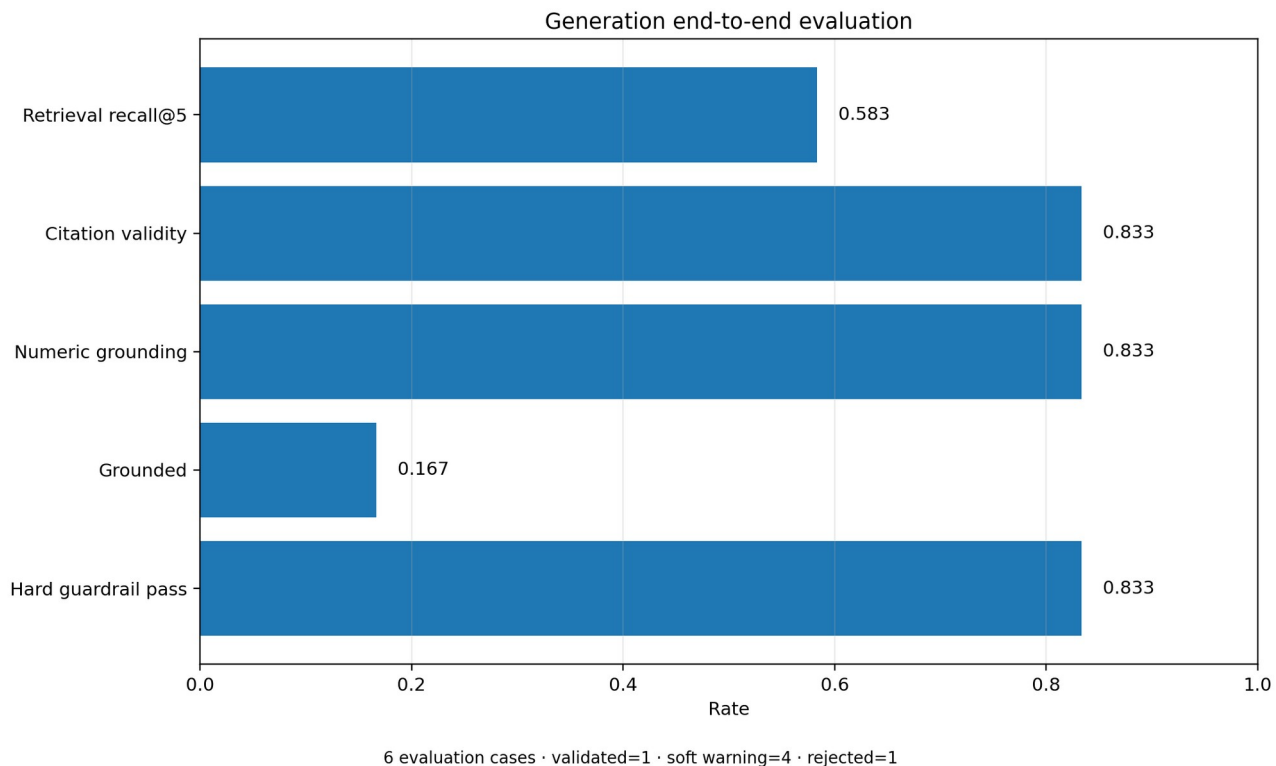


Figure 7. End-to-end generation and validation metrics across six cases.

Validation outcomes

- 1 case: validated
- 4 cases: validated_with_soft_warning
- 1 case: rejected

The strict groundedness rate is the clearest limitation in the current evaluation. It was kept visible rather than optimized away or hidden. The result also demonstrates why the project separates deterministic hard checks from semantic groundedness: citation and numeric validation can succeed even when the entailment validator remains uncertain.

The rejected case is particularly useful from an engineering perspective. The pipeline had retrieved relevant evidence, but the generated answer failed citation and numeric-grounding checks; instead of exposing the candidate answer, the system returned the fail-closed message. This validates the intended failure behavior, not just the happy path.

9. Testing and operational validation

The final lightweight test run reported 21 passing tests, 2 deselected integration tests, and no failures. The two warnings were deprecation warnings originating in the FastAPI/Starlette testing dependency stack, not failed project assertions.

- API health endpoint contract
- Structured /query response contract using a lightweight test double
- Empty-input rejection
- Citation validation

- Numeric validation
- FAISS parity
- LlamaIndex retriever parity

In addition to contract tests, a real FastAPI smoke test starts the service, waits for /health, sends an actual POST /query request, and confirms that the response path succeeds. This was executed successfully on the GPU node.

10. Runtime and cluster validation

The canonical implementation was validated on the university DANTZIG/UMH cluster using an NVIDIA A30 24 GB GPU and Python 3.11.11. The working environment included PyTorch 2.11.0+cu128 and Transformers 5.16.1. The generation model is local and is not fetched from the network at runtime.

Scheduler	SLURM
GPU node	g-0
Partition / QoS	GPU / gpu
GPU	NVIDIA A30, 24 GB
Python	3.11.11
PyTorch	2.11.0+cu128
Transformers	5.16.1

During interactive development, backend and frontend processes ran on the GPU node while the browser remained on the local Windows machine. SSH tunneling and VS Code port forwarding were required for ports 8765 (FastAPI) and 8501 (Streamlit). This operational detail is documented because a remote server printing http://0.0.0.0:PORT does not by itself make the service reachable from the local browser.

11. Reproducibility and repository packaging

The repository is designed to keep source code, tests, evaluation scripts, SLURM wrappers, configuration examples, and result JSONs under version control while excluding large model weights, raw PDFs, embeddings, FAISS indexes, virtual environments, logs, and local secrets.

- pyproject.toml defines the Python package and most runtime dependencies.
- .env.example documents the frontend/backend URL contract without storing secrets.
- .gitignore excludes .env files while explicitly allowing .env.example.
- Raw documents and local model weights are excluded from Git.
- Preparation scripts build chunks, embeddings, and the FAISS index.
- Evaluation scripts and corresponding SLURM wrappers reproduce the main retrieval and generation evaluations.

- The technical README documents environment setup, required artifacts, execution, testing, and evaluation commands.

Reproducibility boundary

Because the five PDFs and the local language-model weights are intentionally not redistributed in the repository, a fresh clone cannot produce the complete end-to-end result without restoring those artifacts first. The README therefore documents this requirement explicitly instead of implying that all assets are self-contained.

12. Limitations

- The corpus contains only five research papers.
- The retrieval holdout contains 11 queries; it is useful for regression and comparison but too small for broad performance claims.
- The end-to-end generation evaluation contains six cases.
- Strict semantic groundedness remains the weakest measured generation metric.
- The canonical generator currently requires CUDA.
- Raw documents, model weights, embeddings, and indexes are not distributed in Git.
- The application is not maintained as a 24/7 public service.
- Docker, MLflow, Prometheus, and Grafana were not implemented and are not claimed as project features.

Why these limitations are acceptable for this project

The goal of this portfolio project is to demonstrate a coherent, defensible engineering workflow rather than simulate an enterprise deployment through unused infrastructure. The project therefore prioritizes real retrieval/generation behavior, validation, reproducibility, testing, evaluation, serving, and documented operational execution over adding technologies solely for appearance.

13. What would change at production scale

- Larger and continuously updated document ingestion pipelines.
- Persistent vector infrastructure and explicit index versioning.
- Authentication, authorization, rate limits, and request quotas.
- Dedicated model serving and concurrency controls.
- Distributed or horizontally scalable serving where workload requires it.
- Centralized metrics, traces, and alerting.
- Containerization and CI/CD.
- Load, latency, failure, and security testing.
- Automated evaluation gates before model/index promotion.
- Data governance, document permissions, and auditability for enterprise corpora.

14. Conclusion

The final project demonstrates an end-to-end RAG system rather than an isolated notebook or retrieval experiment. The implemented path includes data preparation, hybrid retrieval, reranking, local GPU generation, provenance-aware response construction, deterministic and semantic validation, API serving, frontend integration, structured logging, tests, evaluation, and real remote-cluster operation.

The most important outcome is not that every evaluation metric is high. The stronger result is that the system exposes measurable behavior, includes explicit failure modes, and preserves evidence of both successful and weak cases. That makes the project technically inspectable and interview-defensible: design choices, results, limitations, and next steps can all be explained from concrete implementation and evaluation evidence.

Appendix A. Key commands

Backend

```
uvicorn llm_rag.api:app --host 0.0.0.0 --port 8765
```

Frontend

```
streamlit run frontend/app.py --server.address 0.0.0.0 --server.port 8501
```

Lightweight tests

```
pytest -q -m "not integration" tests
```

Retrieval evaluation

```
sbatch slurm/evaluate_retrieval_holdout.sbatch
```

Generation evaluation

```
sbatch slurm/evaluate_generation_end_to_end.sbatch
```

Validated interactive GPU allocation on DANTZIG/UMH

```
sinteractive --partition GPU --qos gpu -w g-0 --cpus-per-task=24 --mem=72G --gres=gpu:1
```

Appendix B. Main evidence produced during validation

- Final Streamlit frontend and end-to-end interaction.
- FastAPI Swagger/OpenAPI contract and structured /query response.
- Health endpoint response.
- Passing lightweight test suite.
- Uvicorn request activity.
- Retrieval holdout chart.
- End-to-end generation evaluation chart.
- System architecture diagram.
- GPU runtime evidence.

- Structured JSON logging.
- HTTP 422 input validation.